

Object-oriented Programming (OOP)

Eunbyung Park
Yonsei University



Textbook Chapter 10

Object-oriented Programming

Disclaimer: all slides provided by the course instructor, Copyright 2025, Yonsei University.

These course materials have been produced and published for the purpose of teaching Yonsei University students, so they cannot be used for any other purposes nor be shared with others. The legal responsibility for the violation lies with the actor him/herself.

Magic Method Pronunciation Rules 😊

- Dunder or magic methods in Python are the methods that have two prefix and suffix underbars '_' in the method name
- Dunder stands for ``Double Under (Underbars)''.
- Examples for magic methods:
 - `__init__()`
 - `__add__()`
 - `__str__()`
- Pronouncing the names of magic methods is obviously complicated due to the underbars
- (``underbar underbar ... '')
- The Python community chose to pronounce the double underbars as ``dunder'':
 - ``dunder-init'', ``dunder-add'', ``dunder-str'' etc.
- <https://wiki.python.org/moin/DunderAlias>

Object-oriented programming (OOP) in a nutshell

Object-oriented programming languages such as Python, provide three fundamental features:

- 1) **encapsulation**
- 2) **inheritance**
- 3) **polymorphism**

Together, they support **a paradigm shift** in software development

- from **functions** and passing variables as arguments to functions ...
- ... to **objects** and **calling methods on objects**.



stack made from functions:

```
def getStack():
```

```
    return []
```

```
def push(s,item):
```

```
    s.append(item)
```

```
st = getStack()
```

```
push(st, 0)
```

```
push(st, 1)
```

```
push(st, 2)
```

stack made from object:

```
class Stack:
```

```
    def __init__(self):
```

```
        self.s = []
```

```
    def push(self, item):
```

```
        self.s.append(item)
```

```
st = Stack()
```

```
st.push(0)
```

```
st.push(1)
```

```
st.push(2)
```

stack made from functions:

```
def getStack():  
    return []
```

```
def push(s,item):  
    s.append(item)
```

```
st = getStack()
```

```
push(st, 0)  
push(st, 1)  
push(st, 2)
```

stack made from object:

```
class Stack:
```

```
    def __init__(self):  
        self.s = []
```

```
    def push(self, item):  
        self.s.append(item)
```

```
st = Stack()
```

```
st.push(0)  
st.push(1)  
st.push(2)
```

- To create a stack, we call function getStack().
- Function getStack() returns the new stack (internally represented as a list)

stack made from functions:

```
def getStack():  
    return []
```

```
def push(s,item):  
    s.append(item)
```

```
st = getStack()
```

```
push(st, 0)
```

```
push(st, 1)
```

```
push(st, 2)
```

stack made from object:

```
class Stack:
```

```
    def __init__(self):  
        self.s = []
```

```
    def push(self, item):  
        self.s.append(item)
```

```
st = Stack()
```

```
st.push(0)
```

```
st.push(1)
```

```
st.push(2)
```

- To call function push(), we provide:
- (1) the stack st (2) the item (0) we want to push on stack st

stack made from functions:

```
def getStack():
```

```
    return []
```

```
def push(s,item):
```

```
    s.append(item)
```

```
st = getStack()
```

```
push(st, 0)
```

```
push(st, 1)
```

```
push(st, 2)
```

stack made from object:

```
class Stack:
```

```
    def __init__(self):
```

```
        self.s = []
```

```
    def push(self, item):
```

```
        self.s.append(item)
```

```
st = Stack()
```

```
st.push(0)
```

```
st.push(1)
```

```
st.push(2)
```

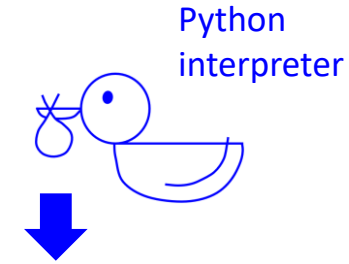
- To create a stack object, we put the name of the **stack class** followed by **()**.
 - Called instantiation.

stack made from functions:

```
def getStack():  
    return []  
  
def push(s,item):  
    s.append(item)  
  
st = getStack()  
  
push(st, 0)  
push(st, 1)  
push(st, 2)
```

stack made from object:

```
class Stack:  
  
    def __init__(self):  
        self.s = []  
  
    def push(self, item):  
        self.s.append(item)  
  
st = Stack()  
  
st.push(0)  
st.push(1)  
st.push(2)
```



- Python creates the stack object and calls `__init__( )` with the new object as its argument for parameter `self`.
- Inside `__init__()`, the stack object is initialized. Then it is assigned to variable `st`.

stack made from functions:

```
def getStack():
```

```
    return []
```

```
def push(s,item):
```

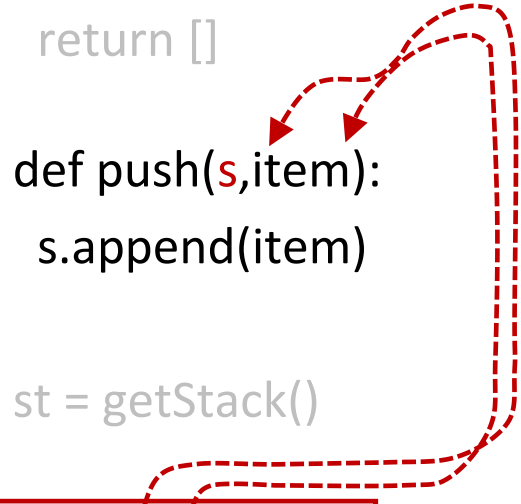
```
    s.append(item)
```

```
st = getStack()
```

```
push(st, 0)
```

```
push(st, 1)
```

```
push(st, 2)
```



stack made from object:

```
class Stack:
```

```
    def __init__(self):
```

```
        self.s = []
```

```
    def push(self, item):
```

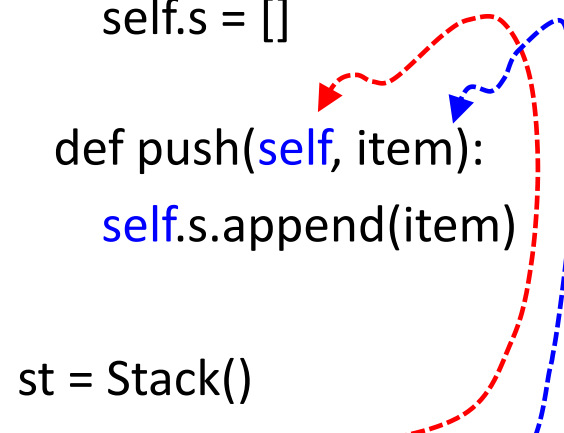
```
        self.s.append(item)
```

```
st = Stack()
```

```
st.push(0)
```

```
st.push(1)
```

```
st.push(2)
```



- The `push()` **method** of the stack object is called using the familiar **dot notation**.
 - Python passes the stack object (`st`) as the argument for the first parameter of the method (`self`).
 - This is very similar to a function call with the function-based stack!

Everything is an object in Python

- Python supports many different kinds of data:
 - integers 1 5 54
 - floats 1.4 54.5
 - strings '54.5'
 - lists [1, 2, 4]
 - dictionaries { 'sun': 'Sunday', 'mon': 'Monday' }
- All are **objects**. Every object has
 - a **type**
 - an internal **data representation**
 - a set of **methods** to work on the object
- An **object** is an **instance** of a type (called **class**)
 - 54 is an instance of the integer class
 - [1, 2, 3, 4] is an instance of the list class

Example: the list class

- The internal **data representation** is usually a linked list.
- Example:
 - `L = [1, 2, 3]`
 - Internally: `L =` 
- How do we work with lists?
- We use the **methods** provided by the **list class** to work on a **list object**.
 - `L.append()`, `L.sort()`, `L.index()`, `L.sort()`
 - `L[x:y]`, `+`, `*`, `...`
- The internal data representation does not concern the user of the list.
 - We say the data representation is **private** to the list.

Defining our own class

keyword starting
class definition

class name

```
class Coordinate:
```

```
# provide attributes here ...
```

Once upon a time, some Python developer already programmed for us:

```
class list:
```

```
...
```

```
class string:
```

```
...
```

```
class float:
```

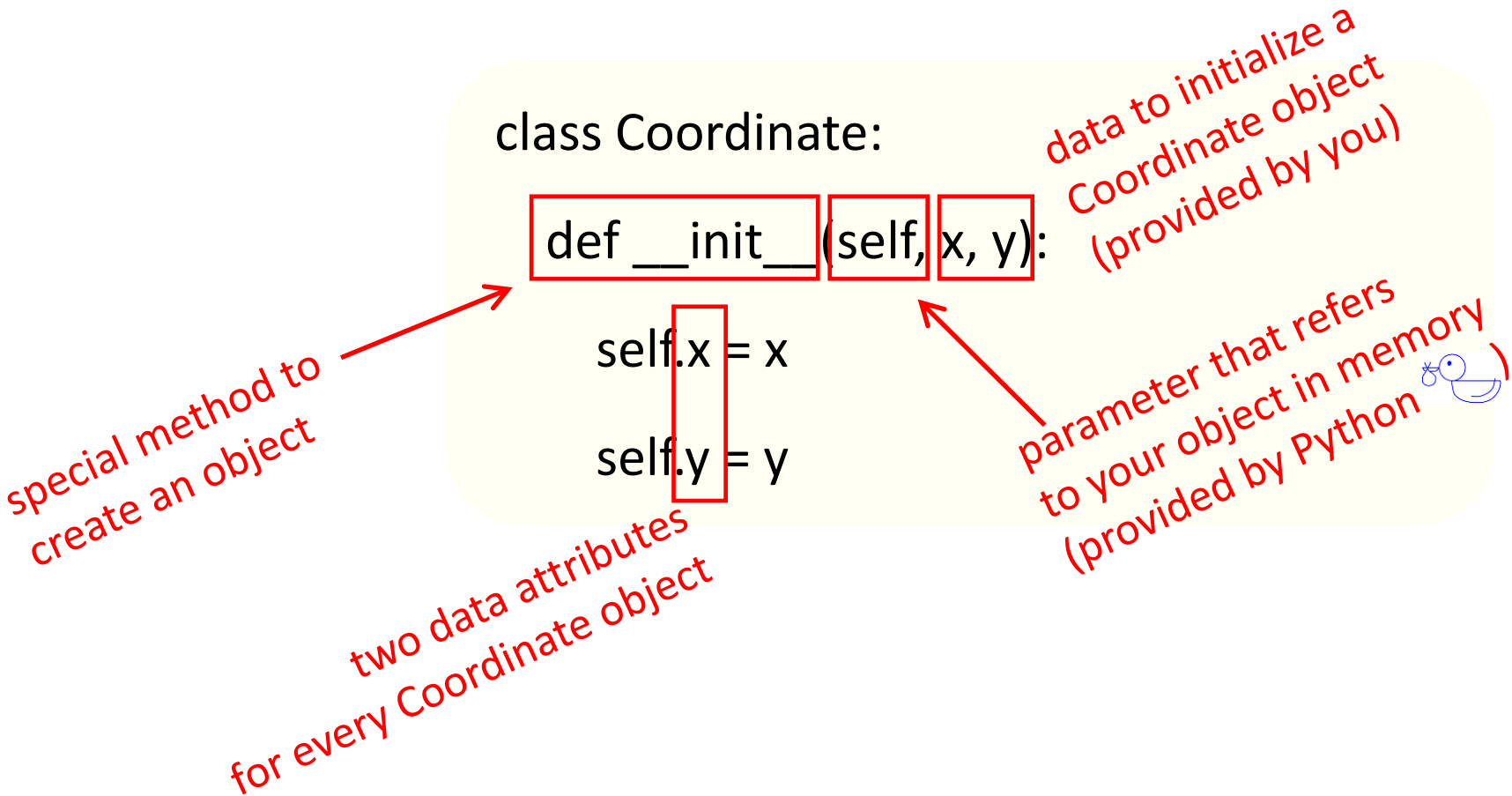
```
...
```

- Similar to functions, all code that belongs to the **class definition** is **indented**.
- The class name (`Coordinate`) is chosen by the programmer.
 - For example, `list` is the name of Python's list class.
 - This is similar to choosing names for functions.
- Our `Coordinate` class **inherits** all attributes of the Python object class (**later in this lecture**).
 - `Coordinate` is a sub-class of `object`.
 - `object` is the super-class of `Coordinate`.

Class attributes

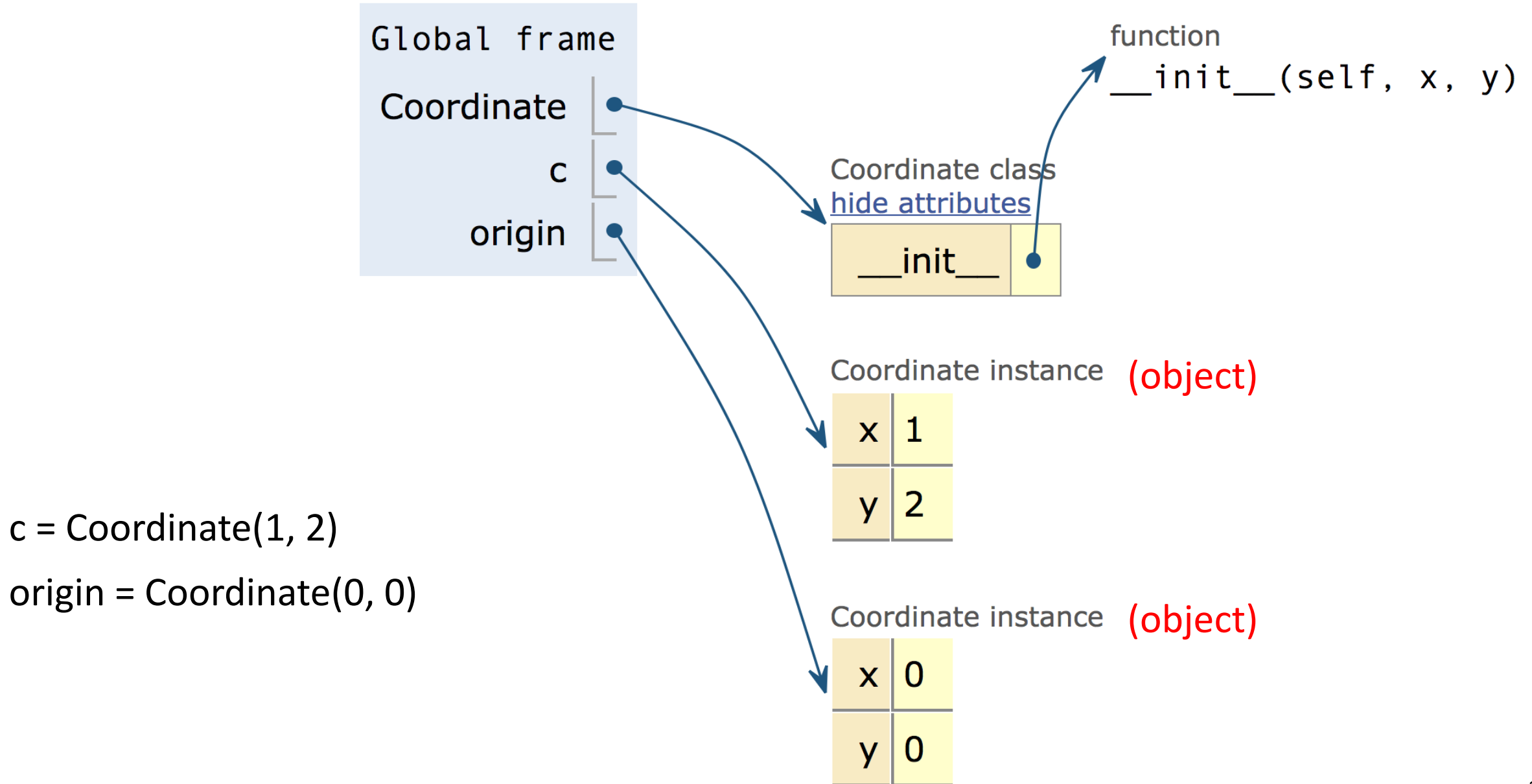
- Attributes: all **data** and **methods** that **belong** to the class.
- **Data attributes:**
 - Objects that belong to the class.
 - Example 1: a 2D coordinate consists of two integer objects
 - Example 2: our stack objects contain a list of ``stacked" values
- **Methods:**
 - Methods are functions that only work for the particular class
 - Methods provide everything we can do with an object
 - Example 1: the append() method of the **list** class
 - Example 2: the push() and pop() methods of our **stack** class
 - Example 3: we can define a distance between two **2D coordinate** objects

Code the construction of class instances



- First step: define **how to create an object** of our class.
- The **special method** called `__init__` must be provided to Python to initialize all data attributes of an object.

Observe the sequence of steps in Python Tutor!



Creating an object (= an instance of a class)

```
c = Coordinate(1, 2)
```

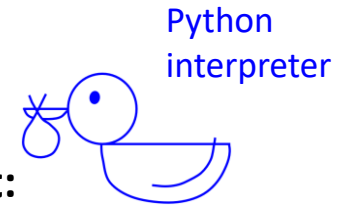
Create new object
of type `Coordinate` and
pass values 1 and 2 to `__init__`

Step 1:

- Python allocates `Coordinate` object in memory.
- The object is initially **empty**.
 - Data attributes are **not** initialized yet!

```
class Coordinate:  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y
```

Coordinate object:

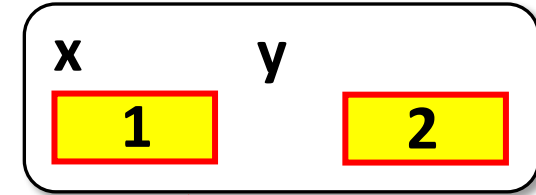


c = Coordinate(1, 2)

Argument for
parameter x
of `__init__`

Argument for
parameter y
of `__init__`

Coordinate object:



Step 2:

- Python calls your `__init__` method
 - Your `__init__` method **initializes the object**
- self** contains reference to the object!
- x** and **y** contain argument values that you provided

```
class Coordinate:
```

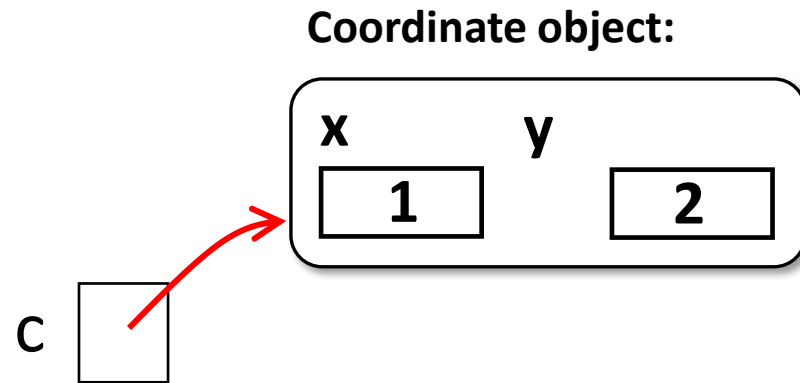
```
    def __init__(self, x, y):
```

```
        self.x = x
```

```
        self.y = y
```

Assignment to data attributes
of object

c = Coordinate(1, 2)



Step 3:

- Python assigns variable `c` to the initialized `Coordinate` object.

```
class Coordinate:  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y
```

Creating an instance of a class

```
c = Coordinate(1, 2)
```

```
origin = Coordinate(0, 0)
```

```
print(origin.x)
```

Use dot notation to
access attribute of instance

- Data attributes of an instance are called **instance variables**.
- You do not have to provide an argument for **self parameter**, Python does it automatically.
 - See previous slides.

Methods of a class

- A **method** is a **class attribute**. Like a function that only works with this particular class.
- Python always passes the object as the first argument to a method.
 - The `self` parameter is used for this purpose.
- Convention to use `self` as the name of the first parameter of all methods.
- The dot `."` operator is used to access all class attributes
 - data attributes
 - methods

Adding a method to a class

class Coordinate:

```
def __init__(self, x, y):
```

```
    self.x = x
```

```
    self.y = y
```

```
def distance(self, other):
```

```
    x_diff = (self.x - other.x)**2
```

```
    y_diff = (self.y - other.y)**2
```

```
    return (x_diff + y_diff)**0.5
```

refers to this Coordinate object 
another parameter to the method

dot notation refers to
object's attributes

- Except the self parameter and the dot notation, methods work like functions.
 - Receive arguments → compute → return values to caller.

Calling a method (2 ways)

```
def distance (self, other):
```

```
...
```

method definition

Call on object:

- (= conventional way)

```
c = Coordinate(4, 3)
```

```
orig = Coordinate(0, 0)
```

```
x = c.distance(orig)
```

↑
object on which
method is called
(represents self)

↑
name of
method to call

↑
argument(s) except self

Call through class name:

```
c = Coordinate(4, 3)
```

```
orig = Coordinate(0, 0)
```

```
x = Coordinate.distance(c, orig)
```

↑
name of class

↑
name of method
to call

↑
object on which
method is called
(represents self)

What is **self** doing, anyway ?

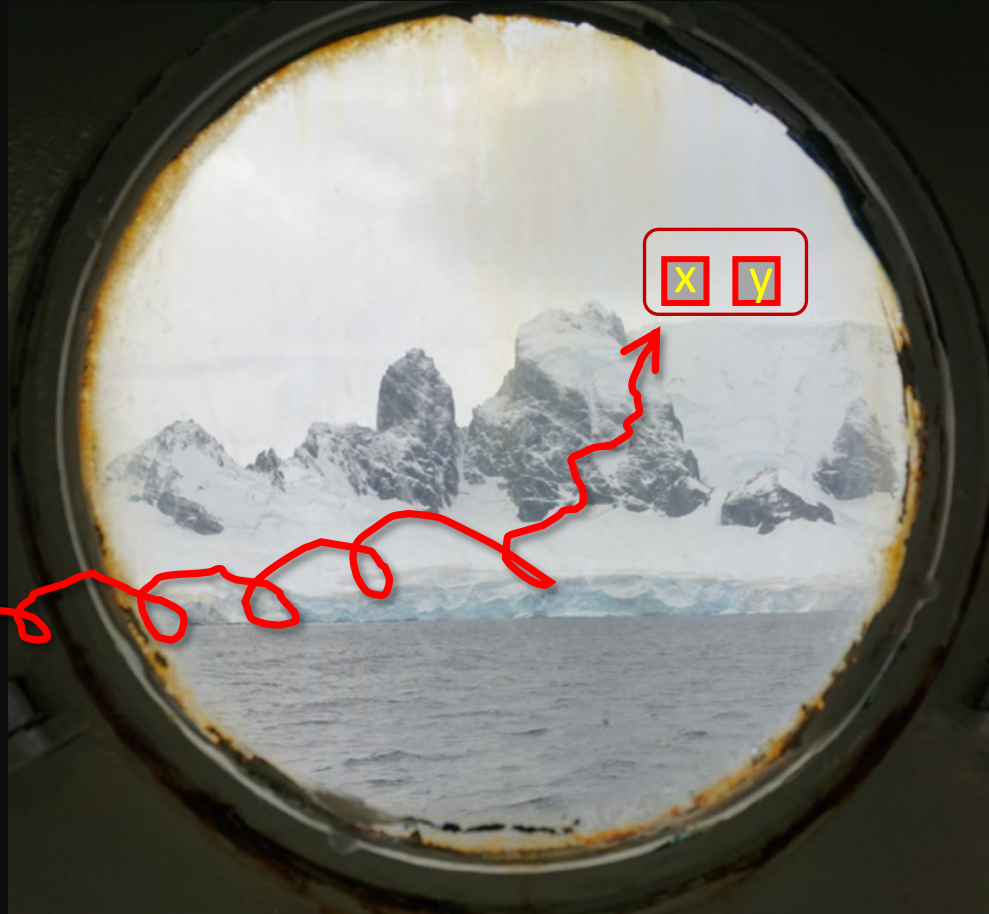
- Python keeps your method's code in a different place from your instance's data.
- The **self** parameter is the lifeline that Python provides to your method to refer to the instance's data 😊



```
def __init__(self, x, y):
```

```
    self.x = x
```

```
    self.y = y
```



- Think of a method as a function with an additional parameter (**self**)

Printing the representation of an instance

```
c = Coordinate(4, 3)
```

```
print(c)
```

```
<__main__.Coordinate object at 0x02215E30>
```

- Python's default print representation of a class instance is **un-informative**.
- Define a **__str__** method for your class!
- Python will call the **__str__** method when your class instance shall be printed.
- Representation is **under your control**. For example, if we want:

```
print(c)
```

```
<4,3>
```



see next slide on how to achieve this

Adding `__str__` method to class

```
class Coordinate:
```

```
    def __init__(self, x, y):
```

```
        self.x = x
```

```
        self.y = y
```

```
    def distance(self, other):
```

```
        x_diff = (self.x - other.x)**2
```

```
        y_diff = (self.y - other.y)**2
```

```
        return (x_diff + y_diff)**0.5
```

```
    def __str__(self):
```

```
        return '<'+str(self.x)+','+str(self.y)+'>'
```

*name of
special method*

*return string
representation*

Special methods for arithmetic and relational operators

- `+`, `-`, `==`, `*`, `/`, `<`, `>`, `len()` and many others
- <https://docs.python.org/3/reference/datamodel.html#basic-customization>

- Define them with double underscores before and after:

<code>__add__(self, other)</code>	$\rightarrow$	<code>self + other</code>
<code>__sub__(self, other)</code>	$\rightarrow$	<code>self - other</code>
<code>__eq__(self, other)</code>	$\rightarrow$	<code>self == other</code>
<code>__lt__(self, other)</code>	$\rightarrow$	<code>self < other</code>
<code>__len__(self)</code>	$\rightarrow$	<code>len(self)</code>
...		

- Assume `__add__()` defined for our `Coordinate` class.
 - Then **infix notation** can be used to add `Coordinate` objects:

```
f = Coordinate(1, 2)
```

```
g = Coordinate(3, 5)
```

```
h = f + g
```

Equivalent to:

```
h = f.__add__(g)
```

Example: Fractions

- Create a **new class** to represent a number as a fraction
 - Examples: $1/2$, $2/5$, ...
- The **internal representation** consists of two integers
 - numerator
 - denominator
- The **methods** of the `Fraction` class are the **interface** to the class.
- The interface defines **how to interact** with `Fraction` objects:
 - create `Fraction` objects
 - multiply, add
 - print the representation
 - convert to float
- See the code in `ch10_combined.py`

Implementing the Class

versus

Using the Class

- Write code from two different perspectives:

Implementing a new type with a class

- **define** the class
- define **data attributes**
 - (What is the object)
- define **methods**
 - (How to use the object)

Content of this lecture 😊

Using instances (objects) of the new class in code:

- create **instances** of the class
- **call methods** on the instances
 - doing **operations** with your objects

What you did until now in this course 😊

Class Definition

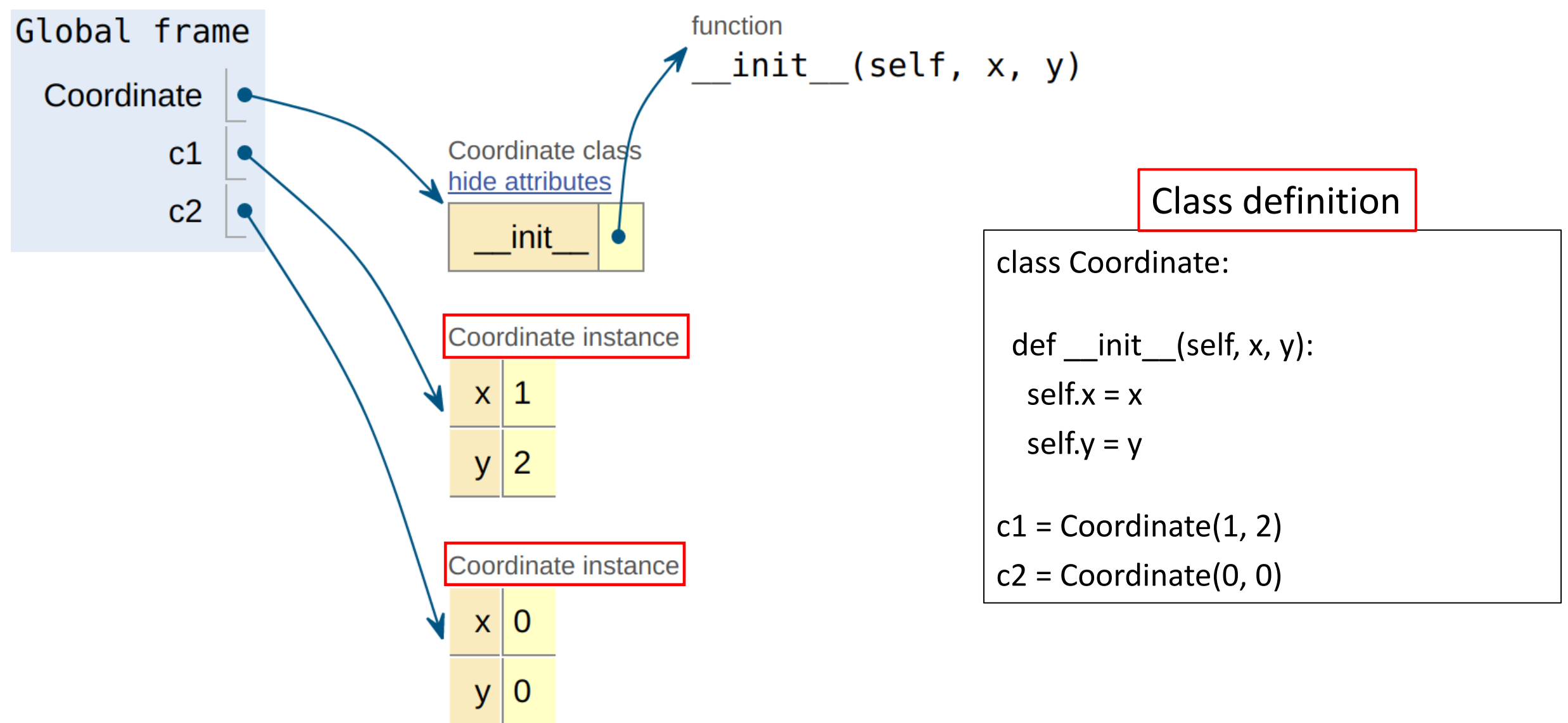
versus

Instance of a Class

- Class name is the type
`class Coordinate:`
- Class is defined in a generic way
 - In class definition, use `self` to refer to **particular instance**
`(self.x - self.y)**2`
 - `self` is a parameter to methods of the class
- Class defines data layout and methods
common across all instances

- Instance is one specific object
`coord = Coordinate(1, 2)`
- Data attribute values different across instances
`c1 = Coordinate(1, 2)`
`c2 = Coordinate(0, 0)`
- `c1` and `c2` have different data attribute values `c1.x` and `c2.x`, because they are different objects
 - See example on next slide
- Each instance shares the **structure** of the class!

Example: Class definition versus class instances



How to define a class (recap)

keyword starting
class definition

class name

parent class

parameter to refer to an instance
of the class
data to initialize a Fraction
object

```
class Fraction(object):
```

```
def __init__(self, n, d):
```

```
    self.num = n
```

```
    self.denom = d
```

special method
to create an instance

```
somefrac = Fraction(1, 3)
```

a Fraction instance

becomes
self.num
of Fraction instance

becomes
self.denom of
Fraction instance

Getter and setter methods

```
class Fraction(object):
```

```
    def __init__(self, n, d):
```

```
        self.num = n
```

```
        self.denom = d
```

```
    def getNumerator(self):
```

```
        return self.num
```

```
    def getDenominator(self):
```

```
        return self.denom
```

```
    def setNumerator(self, newNumerator):
```

```
        self.num = newNumerator
```

```
    def setDenominator(self, newDenominator):
```

```
        self.denom = newDenominator
```

getters

setters

- **Getters** and **setters** should be used **from outside of class** to **access** data attributes.

Instances and the dot notation

- **Instantiation** creates an **instance of an object**:

```
f = Fraction(1, 2)
```

- **Dot notation** allows to access attributes (data and methods)
- **From outside the class**, it is **recommended** to access data attributes through getter and setter methods
 - Inside the class, direct access is ok, because it is your (the class implementor's) matter (see Slide #4)

```
f.num
```

- Direct access from outside to data attribute
- Allowed but **not recommended** 😞
 - But see properties (following slides)

```
f.getNumerator()
```

- Access to data attribute through getter
- **Recommended!** 😊

Encapsulation and Information Hiding

- The author of a class definition may **change data attributes**.

Ex.: **change** names of instance variables

```
class Fraction(object):
```

```
    def __init__(self, n, d):
```

```
        self.n = n
```

```
        self.d = d
```

```
    def getNumerator(self):
```

```
        return self.n
```

```
...
```

Changed data attribute names

- num → n
- denom → d
- Will break user-code that uses previous names! ☹
- Use getters and setters instead!

- When **directly accessing data attributes** from outside of the class, the code will then **break**. (see the example on the previous slide)

- Use getters and setters instead: **f.getNumerator()** not **f.num**
 - Good programming style
 - Easy to maintain code

Python not good at encapsulation 😞

- Python allows you to **read** data from outside of the class definition:

```
print(f.denom)
```

- Allows you to **write** to data from outside of class definition:

```
f.denom = 0
```

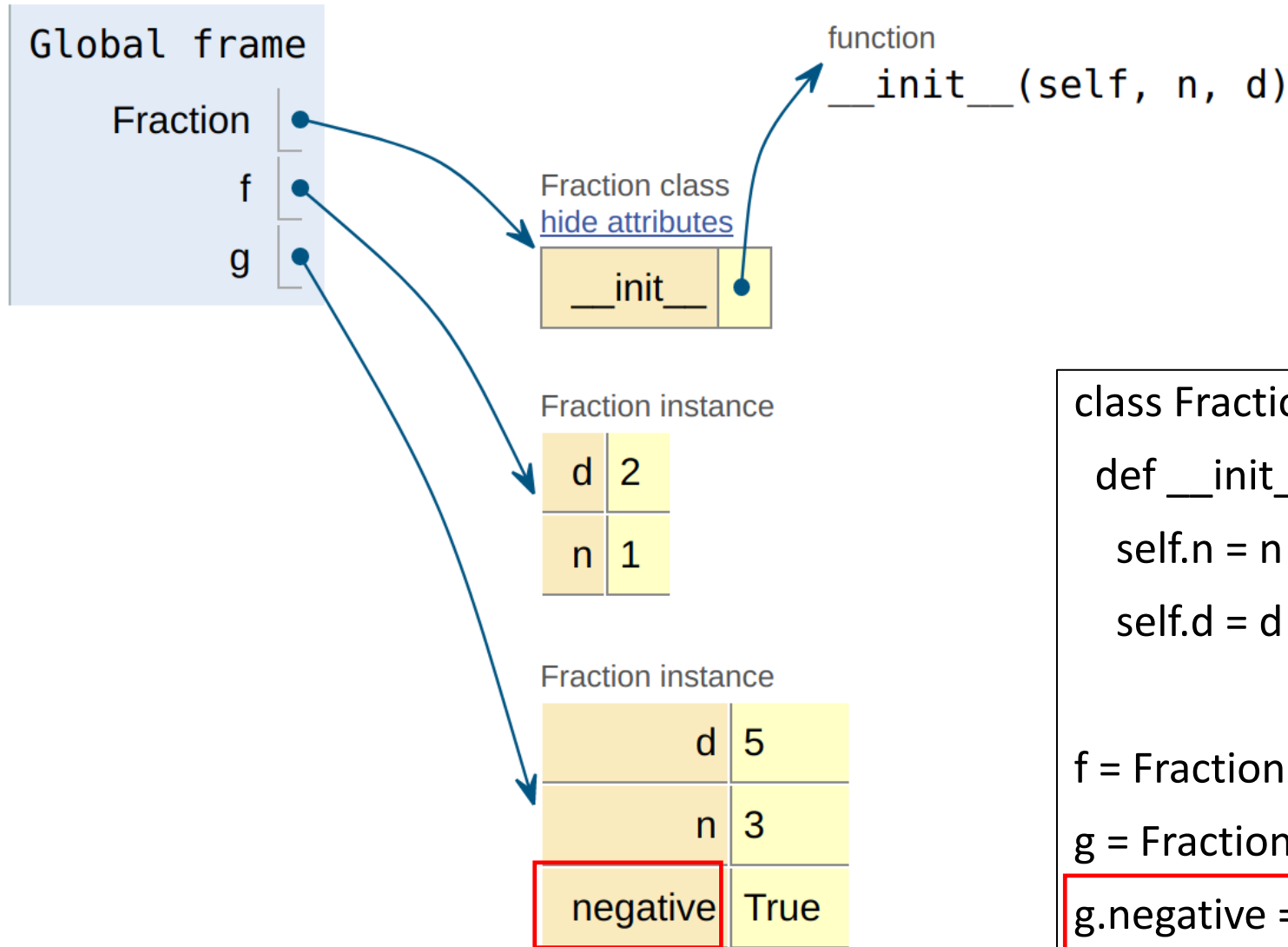
- Allows you to **create** data attributes for an instance from outside of the class definition:

```
f.negative = True
```

- Object now contains additional Boolean attribute!
- See example on next slide

- **Generally**, this is considered **not good programming style**.
- **The Python language** provides properties (see next slides) to fix the first two of the above issues.

You shall not add attributes from outside of a class



```
class Fraction(object):  
    def __init__(self, n, d):  
        self.n = n  
        self.d = d
```

```
f = Fraction(1, 2)  
g = Fraction(3, 5)
```

```
g.negative = True
```



Private Data Attributes and Methods

```
class Fraction(object):  
  
    def __init__(self, n, d):  
        self.__n = n  
        self.__d = d  
  
    def __MyMethod(self): pass
```

- By prefixing a data or method attribute by double underbars (__), they become **private attributes**.
- Private attributes cannot be accessed by their name from outside of the class.
 - See next slide.

```
x = Fraction(1, 2)  
  
x.__n = 0 # does not set numerator
```

Name Mangling of Private Attributes

- Python automatically changes the names of private attributes to begin with a single underbar, followed by the class name.
 - This is called **name mangling**.
- Example: `__d` → `_Fraction__d`
- Unless the variable or method is accessed via its mangled name (from outside), it will not be found.
- Meant to prevent **unintentional** access of private attributes of a class.

Pythonic: Encapsulation by Properties (1)

```
class B:  
  
    def __init__(self, x):  
        self.__x = x  
  
    def get_x(self):  
        return self.__x  
  
    def set_x(self, x):  
        self.__x = x
```

```
m = B(10)  
n = B(12)  
  
m.set_x(m.get_x() + n.get_x())
```



- Getters and setter get **tedious** quickly.
- They **get into our way** to access data attributes ``fluently''.
- Wouldn't it be nicer to use a **public attribute** for x?

```
class B:  
  
    def __init__(self, x):  
        self.x = x
```

```
m = B(10)  
n = B(12)  
  
m.x = m.x + n.x
```



Pythonic: Encapsulation by Properties (2)

```
class B:  
  
    def __init__(self, x):  
        self.x = x
```

your class

```
from the_b import B  
  
m = B(10)  
m.x = -1
```

users of your class

Story:

- Let's assume you chose to use a public data attribute for your class, and your colleagues used your class a lot in their code.
 - By happily accessing the **public** attribute.
- Suddenly your manager asks you to change class B so that attribute x can only be positive (negative values shall be automatically set to zero).
 - Ooops. All legacy code needs to be changed?
 - Shouldn't you have used getters and setters?
- Python has an elegant solution (properties), see next slide.

Pythonic: Encapsulation by Properties (3)

```
class B:  
  
    def __init__(self, x):  
        self.x = x
```

```
@property  
def x(self):  
    return self.__x
```

```
@x.setter  
def x(self, x):  
    if x < 0:  
        self.__x = 0  
    else:  
        self.__x = x
```

data attribute
(hidden)

- The method used for **reading** a value is marked by **@property**
- The **name** of the method is the name of the **public attribute** (x).
- This method becomes the **getter** for attribute x.
- Python will replace every read access to x by a function call to getter x:

```
m = B(10)  
print(m.x) # works! 😊
```

calling getter x

- The actual data attribute has become **__x**.

Pythonic: Encapsulation by Properties (4)

```
class B:  
  
    def __init__(self, x):  
        self.x = x  
  
    @property  
    def x(self):  
        return self.__x
```

```
@x.setter  
def x(self, x):  
    if x < 0:  
        self.__x = 0  
    else:  
        self.__x = x
```

data attribute

- The method that becomes the **setter** is marked by **property @x.setter**
 - The x refers to the public attribute that is set.
- Python will replace every write access to x by a function call to x.setter:

```
m = B(10)  
# Set to 0 by x.setter:  
m.x = -1 # m.x now 0
```

calling setter x.setter

- The actual data attribute has become __x.

Pythonic: Encapsulation by Properties (4)

```
class B:
```

```
    def __init__(self, x):
```

```
        self.x = x
```

```
    @property
```

```
    def x(self):
```

```
        return self.__x
```

```
    @x.setter
```

```
    def x(self, x):
```

```
        if x < 0:
```

```
            self.__x = 0
```

```
        else:
```

```
            self.__x = x
```

calling setter x.setter

data attribute

- The method that becomes the **setter** is marked by property **@x.setter**
 - The x refers to the public attribute that is set.
- Python will **replace** every **write access** to x by a function call to x.setter.
- The setter is also called from __init__:

```
m = B(-100)
# Set to 0 by x.setter
print(m.x) # 0
```

- The actual data attribute has become __x.

Pythonic: Encapsulation by Properties (5)

- **Properties** give you the best of both worlds:
 - ``Look and feel'' of public data attributes, flexible access:
 - Can write: `m.x = m.x + n.x`
 - **Full control** from inside the class and **strong encapsulation**:
 - Each access to the data attribute (from inside and outside of class) is ``intercepted'' by Python and ``re-routed'' to a function call to the getter/setter.
 - The getter/setter functions are under control of the class implementer and can be changed programmatically in any way.
 - The programmer decides the actual data attribute(s).
 - Was `__x` in the previous example, but can be any name except the name of the getter/setter (`foo`, `bar`, `_bar`, `__bar`, ...)
 - Does not have to be a single variable.- There is a growing trend with programming languages to ease the strict getter/setter encapsulation mechanism of private attributes.

Plain data

versus

Encapsulation

- Instead of programming a Fraction class, we could have used plain two-element lists to represent fractions:

```
x = [1, 2]  Fractions as  
            plain lists
```

```
y = [3, 5]  
z = [x[0] * y[0],  
     x[1] * y[1]]  
print(z[0])  
l = [[1,2], [3,4], [1,7]]
```

- Data representation **hard to memorize**
 - is x[0] the numerator?
 - is x[1] the denominator?
- **Data un-protected** from wrong use
- **Messy** because of lack of names
 - Easy to make mistakes

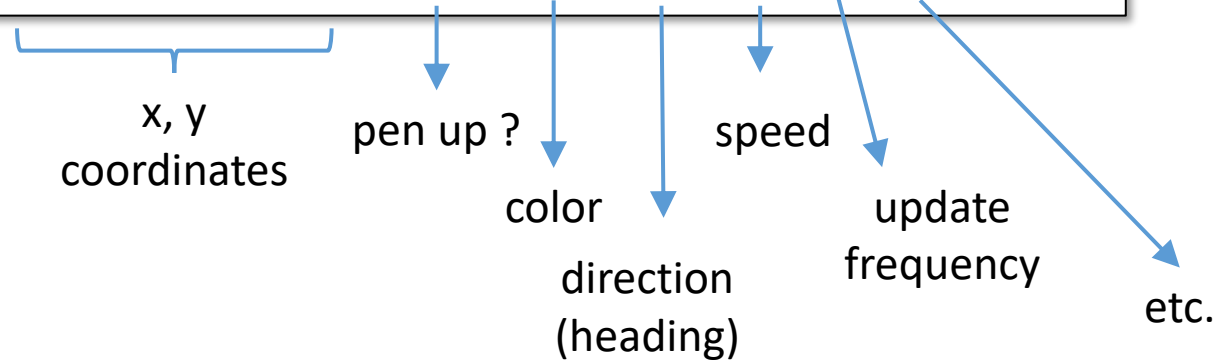
```
x = Fraction(1, 2)  
y = Fraction(3, 5)  
z = x * y  
print(z.getNumerator())  
l = [Fraction(1,2),  
     Fraction(3,4),  
     Fraction(1,7)]
```

- **Easy to read**
 - **Names** are ``speaking'' to the human programmer
- Data **encapsulated** inside of the Fraction object
- **Convenience methods** can be provided (+, -, __str__, ...)

Complex Data and Information Hiding

- The **complexity** of **common classes** such as the Python turtle can get **overwhelming** quickly when provided as flat data such as a list:

```
myTurtle = [[10.0, 210.0], True, 2, 270, 0, 10, ...]
```

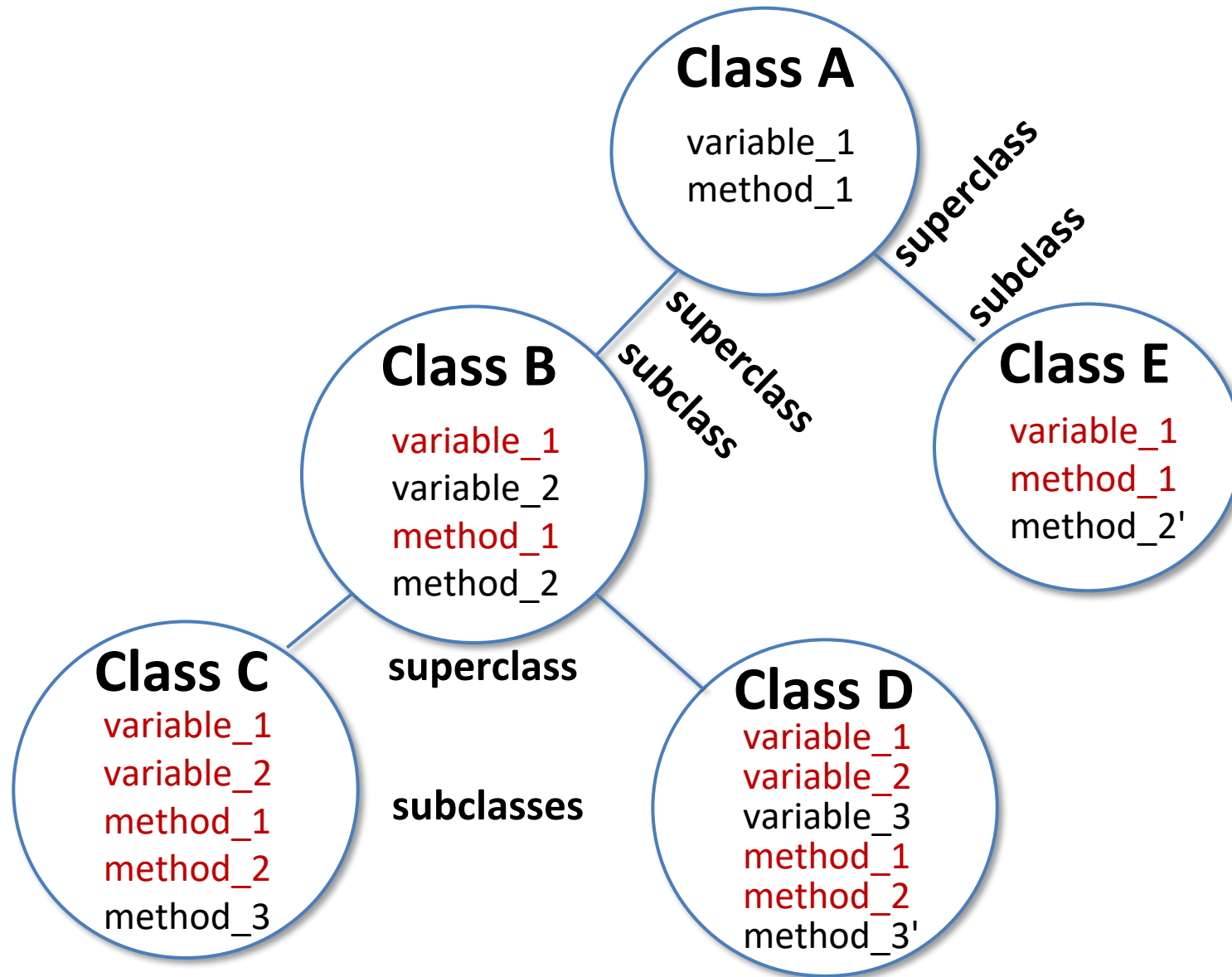


- As a client of the Turtle class, we don't have to know about a turtle's internal data representation.
 - And we should not know! If a new Python release changes the turtle's data representation, then all client code that directly manipulates turtle data will break. ☹️
- It is therefore very important that the turtle class **hides** all its data from the client.
 - The client can only call methods to affect a turtle object.
 - This concept is called **encapsulation**.
 - It is a fundamental concept of OOP.
 - Next we're going to study **inheritance**, another fundamental concept of OOP.

Inheritance

- A class can **inherit** methods and instance variables from another class.
 - The inheriting class is the **child class** (or **subclass**).
 - The class inherited from is the **parent class** (or **superclass**).
- Inherited methods and instance variables **become available in the child** as if the child itself had defined them.
- Superclasses may themselves inherit from other classes, resulting in a **hierarchy** of classes.
- Purpose: **extend** a class with **new functionality**.
 - A form of **code reuse**.

Inheritance Across a Class Hierarchy



The items in **red**
are inherited from
the superclass.

- Person inherits from object
- In Python, everything is an object
- Class ``object'' implements basic operations required by all classes



```
class Person(object):  
    def __init__(self, name):  
        self.name = name  
        self.birthday = None  
    def getName(self):  
        return self.name  
    def setBirthday(self, birthDate):  
        if type(birthDate) != datetime.date:  
            raise ValueError("require datetime.date")  
        self.birthday = birthDate  
    def getAge(self):  
        if self.birthday == None:  
            raise ValueError("Birthday not set")  
        return (datetime.date.today() \  
                - self.birthday).days  
    def __str__(self):  
        return self.name
```

*Check that birthday is of
correct type.*

*Check that birthday
has been set already*

```
class YonseiPerson(Person):
```

Inherit from Person class

```
    def __init__(self, name, ID):
```

```
        Person.__init__(self, name)
```

```
        self.ID = ID
```

```
    def getID(self):
```

```
        return self.ID
```

```
    def __str__(self):
```

```
        return self.name + " " + str(self.ID)
```

Call __init__ of superclass

Add ID data attribute

- YonseiPerson is a **subclass** of our Person class.
- It **inherits** all properties of the **superclass**
 - all instance variables
 - all methods
- The YonseiPerson `__init__` method calls the `__init__` method of the superclass
 - To initialize the instance variable(s) inherited from the superclass (name)
- Then the YonseiPerson `__init__` method initializes its own instance variable (ID).

```
class YonseiPerson(Person):  
    def __init__(self, name, ID):  
        Person.__init__(self, name)  
        self.ID = ID  
    def getID(self):  
        return self.ID  
    def __str__(self):  
        return self.name + " " + str(self.ID)
```

Add getter for ID data attribute

- YonseiPerson is a **subclass** of our Person class.
- It **adds new functionality** with the **getID()** method to retrieve the ID of a YonseiPerson object.

Overriding a method from the superclass

```
class YonseiPerson(Person):  
    def __init__(self, name, ID):  
        Person.__init__(self, name)  
        self.ID = ID  
    def getID(self):  
        return self.ID  
    def __str__(self):  
        return "YU's" + self.name
```

Override the `__str__` method to
give YonseiPersons a "special"
touch 😊

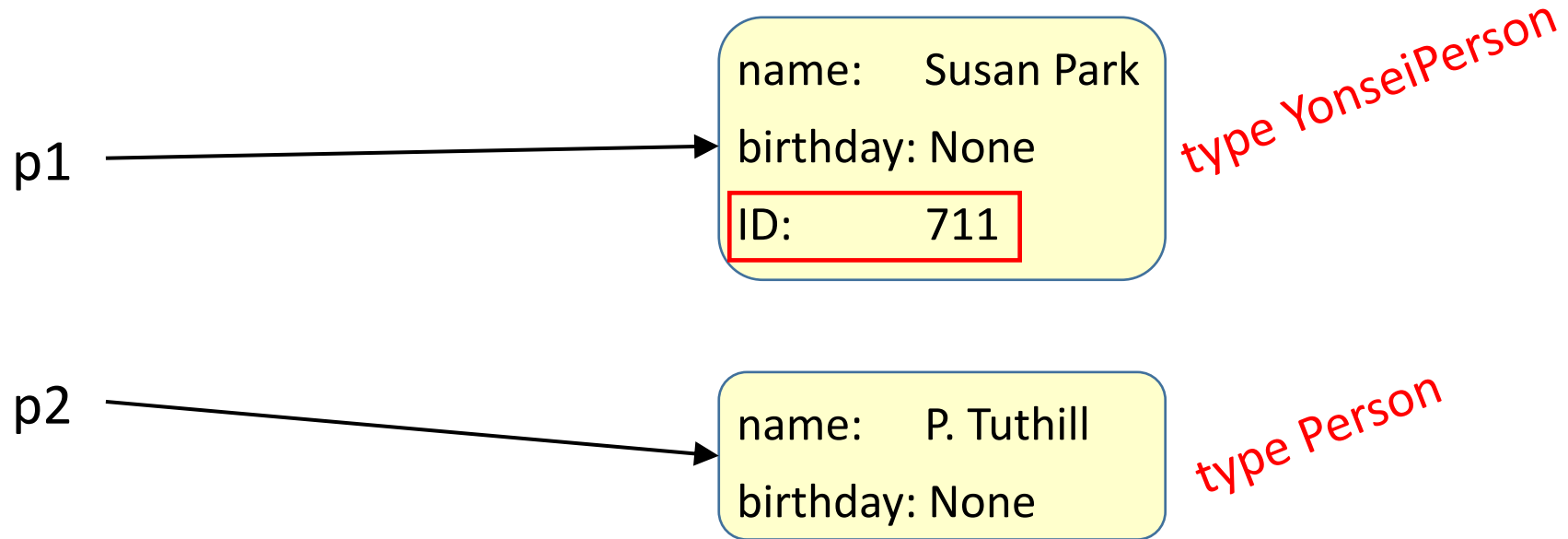
- A subclass can **re-define** a method from the superclass.
- We say the subclass **overrides** the method from the superclass.

```
p1 = Person('Jay')  
p2 = YonseiPerson('Jiin', 1)  
print(p1, 'and', p2)
```

Output:
Jay and YU's Jiin

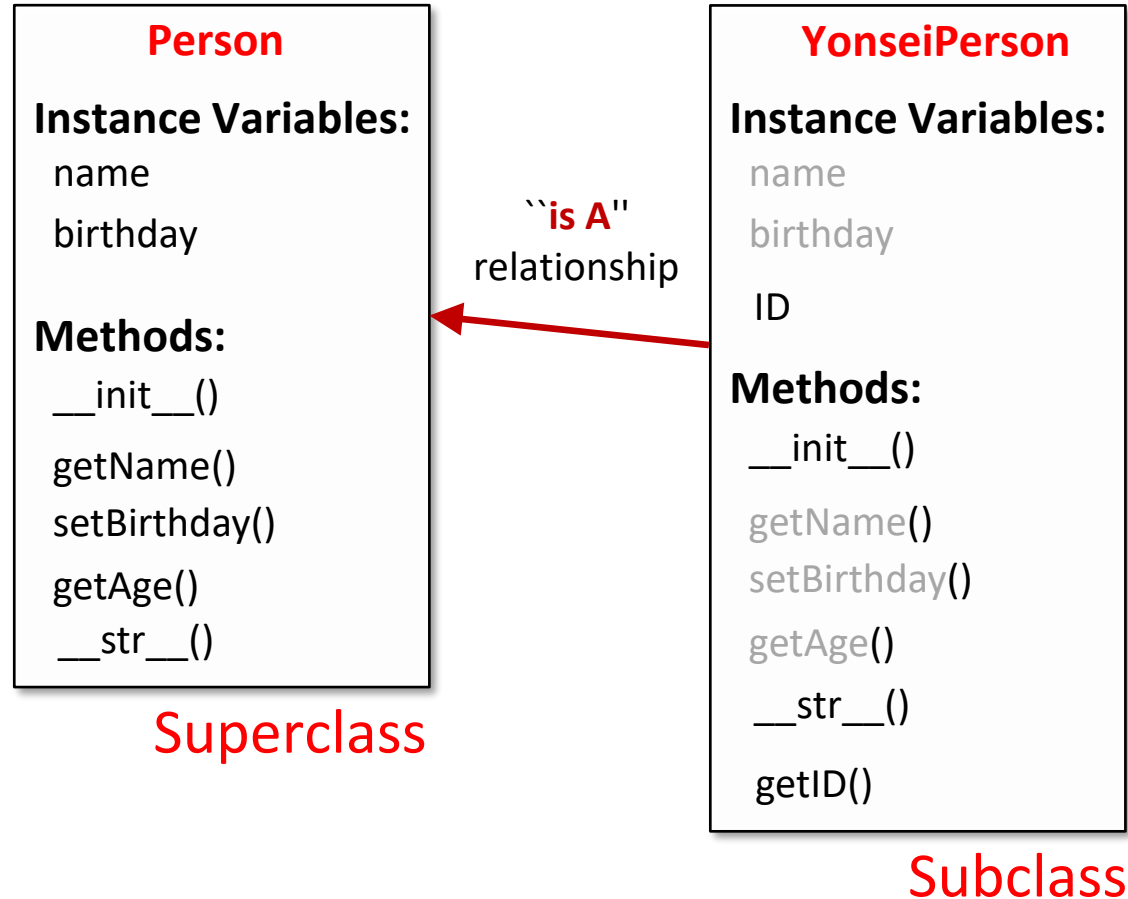
```
p1 = YonseiPerson('Susan Park', 711)
```

```
p2 = Person('P. Tuthill')
```



- YonseiPerson has an ID data attribute **in addition** to all data attributes inherited from the Person class.

Subclasses and the ``is A'' relationship

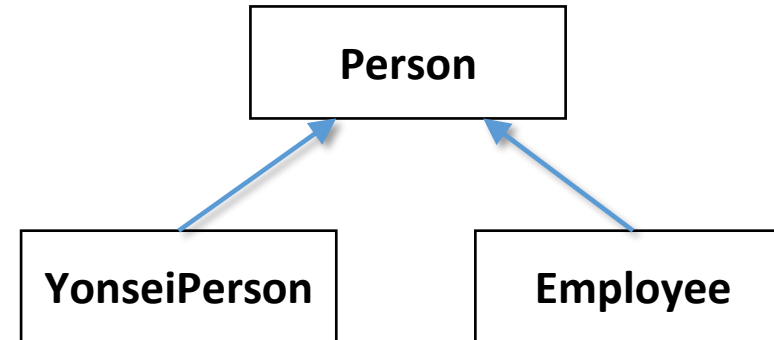


- YonseiPerson **inherits** everything from its Superclass.
 - Shown in gray.
- It **adds** member variables and methods.
 - But it **cannot subtract** (impossible with OOP).
- The subclass is thus a **superset** of the variables and methods of the superclass.
 - Has everything of the Superclass and more.
- ``**Is A**'' relationship: every YonseiPerson is a Person.
 - has everything the Person has
 - (but not vice versa!)

- Imagine a function that expects a Person as its argument.
- You can provide a YonseiPerson instead.
 - (Because of the **Is A** relationship of subclasses to superclasses).

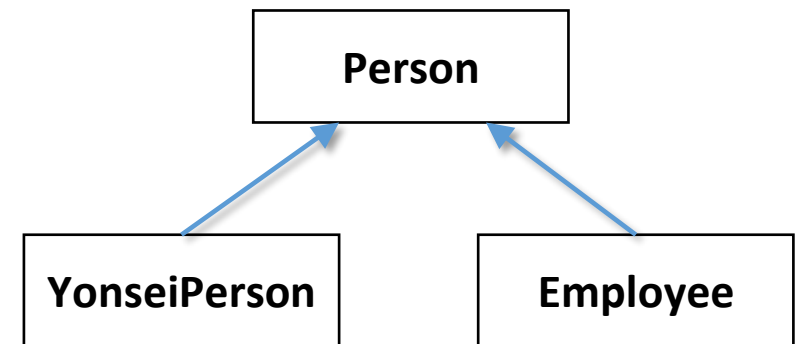
```
class Employee(Person):  
    '''Employee is a descendant of class Person.  
    An employee has a department and a salary.'''  
    def __init__(self, name, dept, salary):  
        Person.__init__(self, name)  
        self.dept = dept  
        self.salary = salary  
    def getDepartment(self):  
        return self.dept  
    def getSalary(self):  
        return self.salary
```

- Our class hierarchy now consists of three classes.
- Each subclass **inherits everything** from the superclass.
- Each subclass **adds its own attributes** (data and methods).




```
class PersonList(object):
    def __init__(self, listname):
        self.listname = listname
        self.persons = []
    def addPerson(self, candidate):
        if not isinstance(candidate, Person):
            raise TypeError('Not a person type')
        if candidate in self.persons:
            raise ValueError('Duplicate person')
        self.persons.append(candidate)
    def removePerson(self, candidate):
        try:
            self.persons.remove(candidate)
        except:
            print(str(candidate) + ' not in ' + self.listname)
    def getYonseiPersons(self):
        ps = []
        for p in self.persons:
            if type(p) == YonseiPerson:
                ps.append(p.getName())
        return ps
```

*True if candidate is
instance of class
Person or of a
subclass of Person*



- The PersonList class is capable of storing any instance of class Person or its subclasses (PersonList is **polymorphic**).

Doboggi-Man Code Reading



- Doboggi-Man is a game written in Python.
- It uses objects to represent its main characters:
 - Dobbogi-Man
 - Dobbogi, ghosts, ...
- The code is small and we'll study it in class as a preparation for the lab.
- **Note:** code reading is an important skill.
 - To understand legacy code
 - For debugging
 - To learn from others

Doboggi-Man Class Hierarchy

